

From Source to Execution: A Statically-Typed Interpreted Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akaradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a statically-typed programming language developed for a programming languages and compilers assignment. The language supports four primitive types — Integer, Float, Boolean, and String — with type inference, arithmetic and boolean expressions, assignment statements, if-then-else and while-loop control flow, function abstraction with value parameter passing, and a built-in `print()` function. The implementation follows a classical compiler pipeline consisting of lexical analysis, recursive-descent parsing, abstract syntax tree construction, static type checking, and tree-walking interpretation. A command-line runner and a desktop user interface are provided so that script files can be loaded, executed, and inspected. The report describes the language grammar, the type inference algorithm, the implementation structure, and the testing used to validate the system.

CONTENTS

ABSTRACT	ii
1 INTRODUCTION	1
1.1 Report Structure	1
2 LANGUAGE DESIGN AND GRAMMAR	3
2.1 Types	3
2.1.1 Integer	3
2.1.2 Float	3
2.1.3 Boolean	3
2.1.4 String	3
2.1.5 Void (internal only)	4
2.2 Static Typing	4
2.3 Token Specification	4
2.4 Context-Free Grammar	5
2.4.1 Program Structure	5
2.4.2 Arithmetic Expressions	5
2.4.3 Boolean Expressions	5
2.4.4 Statements	6
2.4.5 Function Definitions and Calls	6
2.5 Operator Precedence and Associativity	6
3 LEXICAL ANALYSIS AND SYMBOL TABLE	7
3.1 Token representation	7
3.2 Lexer implementation	8

3.3	Symbol table and semantic structure	10
3.4	Role of the symbol table in the pipeline	11
4	RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE	13
4.1	Abstract syntax tree representation	13
4.1.1	Expressions	14
4.1.2	Statements	14
4.2	Parser implementation	15
4.2.1	Statement parsing	15
4.2.2	Expression parsing and precedence	16
4.2.3	Helper functions and lookahead	17
4.3	AST printing	17
4.4	Parser and type checker boundary	18
5	TYPE INFERENCE ALGORITHM	19
5.1	Overview	19
5.2	Type Environment	19
5.3	Inference Rules	19
5.3.1	Literals	19
5.3.2	No Operator Overloading	20
5.3.3	Arithmetic Expressions	20
5.3.4	Boolean Expressions	21
5.3.5	Assignment Statement	21
5.3.6	If-Then-Else	21
5.3.7	While-Loop	21
5.3.8	Function Definition and Call	21
5.4	Type Error Reporting	22
5.5	Example Type Inference Traces	22

6	SYSTEM ARCHITECTURE AND IMPLEMENTATION	24
6.1	Compiler pipeline	24
6.1.1	Entry points and shared pipeline	25
6.2	Project structure	25
6.3	Technology stack	25
6.4	Type checker	26
6.4.1	Scope of the implemented type system	26
6.5	Interpreter	26
6.5.1	Scope of the implemented runtime	27
6.6	Graphical user interface	27
7	TESTING AND VERIFICATION	29
7.1	Test Cases	29
7.1.1	Lexer and Symbol Table	29
7.1.2	Parser and AST	30
7.1.3	Arithmetic Expressions	31
7.1.4	Boolean Expressions	31
7.1.5	Assignment Statements	31
7.1.6	If-Then-Else	31
7.1.7	While-Loop	31
7.1.8	Functions	31
7.1.9	Print	31
7.1.10	Unary Minus	32
7.2	Type Error Detection	32
7.3	Automated Test Suite	32
7.4	Error Handling	33
8	CONCLUSION	34

REFERENCES	36
APPENDIX A: SOURCE CODE	37
APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS	81

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a small statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` statement. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser and abstract syntax tree, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the four pipeline stages — tokens, AST, inferred type table, and execution output — for inspection. An automated regression suite of 54 tests verifies correctness across all stages and language features.

1.1 Report Structure

Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type

inference algorithm and the semantic rules used by the checker. Chapter 6 presents system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language supports four primitive types. These types are sufficient to cover the assignment requirements while keeping the semantics simple and predictable.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 20.5. Float values are used in arithmetic expressions and also arise as the result of any division. Division always produces a Float even when both operands are Integer: for example, `10 / 2` evaluates to `5.0 : Float`. This is an explicit design rule enforced in the type checker.

2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At run-time, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void (internal only)

Void is not a programmer-visible type. The language has exactly four primitive types: Integer, Float, Boolean, and String. Void exists solely as an internal sentinel inside the `LanguageType` enum so that the symbol table can record a well-defined return type for functions that contain no return statement. Programmers cannot write Void in source code.

2.2 Static Typing

The language uses static typing, which means type errors are detected before execution rather than during program execution. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;
2 x = "hello";    (* TypeError: Cannot assign String to variable 'x' of
   type Integer *)
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
INT_LITERAL	0, 10, 42
FLOAT_LITERAL	3.14, 20.5
BOOL_LITERAL	true, false
STRING_LITERAL	"hello"
IDENTIFIER	variable and function names such as x, add
Arithmetic operators	+, -, *, /
Comparison operators	==, !=
Assignment operator	=
Keywords	if, else, while, def, return, print
Delimiters	() { } , ;

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
1 program    -> statement* EOF
2 statement  -> assignment
3           | if_stmt
4           | while_stmt
5           | func_def
6           | print_stmt
7           | return_stmt
8           | expr ';' ;
```

2.4.2 Arithmetic Expressions

```
1 expr       -> term (('+' | '-' ) term)*
2 term       -> factor (('*' | '/' ) factor)*
3 factor     -> '-' factor
4           | INT_LITERAL
5           | FLOAT_LITERAL
6           | STRING_LITERAL
7           | BOOL_LITERAL
8           | IDENTIFIER
9           | IDENTIFIER '(' args? ') '
10          | '(' expr ') ' ;
```

The unary minus production allows negative literals such as -5 and negated variable references such as -x. It is implemented as a right-recursive descent into factor and is desugared internally to `0 - factor`, so all existing type rules for arithmetic apply without modification.

2.4.3 Boolean Expressions

```
1 bool_expr  -> expr '==' expr
2           | expr '!=' expr
3           | BOOL_LITERAL
4           | expr (* fallback: type checker enforces Boolean *)
```

Boolean expressions are only parsed as conditions inside if and while statements. Comparison results are not storable in variables directly; the grammar does not include `bool_expr` in the factor rule. This is consistent with the assignment specification, which describes Boolean

expressions primarily as conditions.

If `bool_expr` parses an expression and finds no following `==` or `!=` operator, the expression is returned as-is and the type checker verifies that its inferred type is `Boolean`.

2.4.4 Statements

```
1 assignment -> IDENTIFIER '=' expr ';'
2 if_stmt    -> 'if' '(' bool_expr ')' block ('else' block)?
3 while_stmt -> 'while' '(' bool_expr ')' block
4 print_stmt -> 'print' '(' expr ')' ';'
5 return_stmt -> 'return' expr ';'
6 block      -> '{' statement* '}'
```

2.4.5 Function Definitions and Calls

```
1 func_def -> 'def' IDENTIFIER '(' params? ')' block
2 params   -> IDENTIFIER (',' IDENTIFIER)*
3 args     -> expr (',' expr)*
```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==</code> , <code>!=</code>	non-associative (condition use only)
2	<code>+</code> , <code>-</code>	left-associative
3 (highest)	<code>*</code> , <code>/</code>	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return, print	keyword-specific token variants
Operators	+, -, *, /, =, ==, !=	PLUS, MINUS, STAR, SLASH, ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (EOF) token. The EOF token is appended after all input is processed,

allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end. Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
8             tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
10    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```
1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }
```

```
1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "+": TokenType.PLUS,
3     "-": TokenType.MINUS,
4     "*": TokenType.STAR,
5     "/": TokenType.SLASH,
6     # ...
7 }
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
KEYWORDS map	Maps reserved words to token types	token type selection
SINGLE_CHAR_TOKENS map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:
6         return self.parent.lookup(name)
7     raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
LanguageType enum	Integer, Float, Boolean, String, Void	Defines the type system
Symbol (base)	Name and declared or inferred type	Base abstraction for entries
VariableSymbol	Variable metadata (including initialization state)	Tracks correct usage before assignment
FunctionSymbol	Parameters and return type	Validates calls and return consistency
lookup(name)	Recursive scope search	Resolves identifiers or reports undefined
SymbolTableError	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     ast_text = ASTPrinter().print(tree)
5     checked_scope = TypeChecker().check(tree)
6     outputs: list[str] = []
7     Interpreter(output_callback=outputs.append).execute(tree)
8     return PipelineResult(
9         tokens=tokens,
10        ast_text=ast_text,
11        checked_scope=checked_scope,
12        outputs=outputs,
13    )

```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed in condition contexts (if/while) through the parser’s boolean-expression path.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (line, column). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
4         diagnostics."""
5     line: int = field(default=0, kw_only=True)
6     column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+', '-', '*', '/', '==', '!=',
6     Example:  a + b    x == 0
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single Program AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```

1 # assignment: IDENTIFIER "=" expr ";"
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr ";"
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")

```

4.2.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles `+` and `-`, `_term` handles `*` and `/`, and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Boolean conditions for `if` and `while` are parsed by `_bool_expr`, which recognises `==` and `!=` after parsing the left-hand arithmetic expression. As a result, equality parsing is condition-focused in the current grammar implementation rather than a general infix operator available in every expression position. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code.

```

1 def _expr(self) -> ASTNode: # +, -
2     ...
3 def _term(self) -> ASTNode: # *, /
4     ...
5 def _factor(self) -> ASTNode: # unary -, literals, identifiers/calls,
6     grouped expr

```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_expr` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

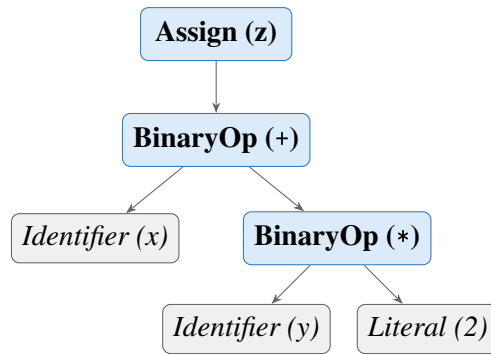


Figure 4.1. AST for $z = x + y * 2$; . Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Boolean conditions for `if` and `while` are parsed through `_bool_expr`, which accepts equality and inequality comparisons but can also return a non-comparison expression node; semantic enforcement that a condition is truly Boolean is deferred to the type checker.

4.2.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token; comparison operators are recognised after parsing the left expression in `_bool_expr`. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure:

Program and Block include statement counts, BinaryOp explicitly shows left and right subtrees, and FunctionCall prints each argument by index. Literal includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_bool_expr` may syntactically accept a non-Boolean expression as a condition node, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \text{Identifier} \rightarrow \text{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Inference Rules

5.3.1 Literals

Literal nodes have direct types:

$$\begin{aligned}\Gamma \vdash 42 &: \text{Integer} \\ \Gamma \vdash 3.14 &: \text{Float} \\ \Gamma \vdash \text{true} &: \text{Boolean} \\ \Gamma \vdash \text{"hi"} &: \text{String}\end{aligned}$$

5.3.2 No Operator Overloading

The language does not support operator overloading. Every operator has a single, fixed meaning determined solely by the operator symbol. Specifically:

- The `+` operator is not overloaded for string concatenation. Applying `+` to a `String` operand is a type error.
- The comparison operators `==` and `!=` are not overloaded for string or boolean equality. Both operands must be arithmetic.
- No mixed Boolean or String arithmetic is permitted.

Because operators have unambiguous types, the compiler can always infer expression types without requiring explicit type declarations from the programmer.

5.3.3 Arithmetic Expressions

The arithmetic operators `+`, `-`, `*`, and `/` accept only numeric operands. If both operands are `Integer`, the result is `Integer`, except for division, which always produces `Float` regardless of the operand types. If at least one operand is `Float` for `+`, `-`, or `*`, the result is `Float` — this is numeric promotion, not operator overloading, because `+` retains a single numeric meaning throughout. Any non-numeric operand causes a type error.

Table 5.1. Arithmetic result types by operand combination

Left operand	Operator	Right operand	Result type
Integer	<code>+</code> , <code>-</code> , <code>*</code>	Integer	Integer
Integer	<code>/</code>	Integer	Float
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Integer	Float
Integer	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Any	any	String or Boolean	TypeError

Unary negation

Unary minus ($-x$) is desugared by the parser into $0 - x$. The type checker therefore applies the standard binary subtraction rule: if x is `Integer`, the result is `Integer`; if x is `Float`, the result is `Float`.

5.3.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Following the language specification, both operators require two arithmetic (`Integer` or `Float`) operands. Comparisons involving `Boolean` or `String` operands are rejected by the type checker.

5.3.5 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma, x : T \vdash x = e$$
$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma \vdash x = e$$

5.3.6 If-Then-Else

The condition of an `if` statement must be `Boolean`. The `then`-block and `else`-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{\text{then}} \quad \Gamma'' \vdash B_{\text{else}} \quad \Longrightarrow \quad \Gamma \vdash \text{if}(c) B_{\text{then}} \text{ else } B_{\text{else}}$$

5.3.7 While-Loop

The condition of a `while` loop must also be `Boolean`. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B \quad \Longrightarrow \quad \Gamma \vdash \text{while}(c) B$$

5.3.8 Function Definition and Call

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the

return statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally rejected in this implementation because they were not required by the assignment and would require additional inference and control-flow analysis.

5.4 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

5.5 Example Type Inference Traces

Consider the following program fragment:

```
1 x = 10;  
2 y = 20.5;  
3 z = x + y;  
4 print(z);
```

The checker infers:

- `x`: `Integer`
- `y`: `Float`
- `x + y`: `Float`
- `z`: `Float`

For a function example:

```
1 def add(a, b) {  
2     return a + b;  
3 }  
4  
5 result = add(2, 3.5);
```

The first function call infers `a`: `Integer` and `b`: `Float`. The expression `a + b` is therefore `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

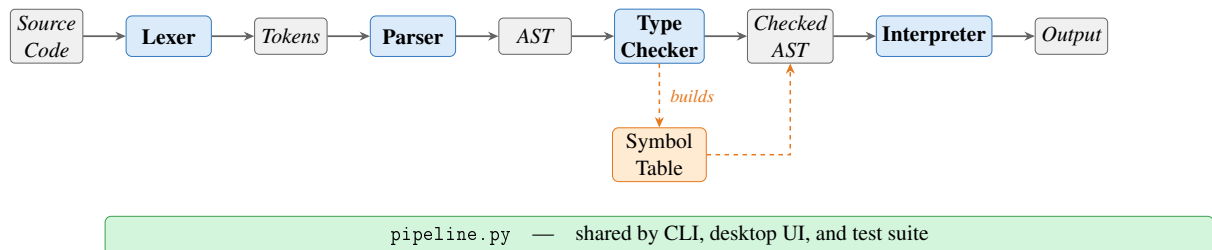


Figure 6.1. Compiler pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, `python3 main.py` runs a script (or built-in sample) and prints the stages in order: tokens, AST text, type table, and execution output. `python3 ui.py` runs the Tkinter workbench: the same pipeline is used internally, with each stage shown in a separate tab so the user can inspect results without altering the underlying flow.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
<code>main.py</code>	Command-line runner for sample code or a script file
<code>ui.py</code>	Desktop UI for editing and running a script
<code>components/tokens.py</code>	Token classes and token categories
<code>components/lexica.py</code>	Lexical analysis
<code>components/ast_nodes.py</code>	AST node definitions
<code>components/parser.py</code>	Recursive-descent parser
<code>components/ast_printer.py</code>	AST renderer for debugging and reporting
<code>components/symbol_table.py</code>	Scoped symbol table and semantic types
<code>components/type_checker.py</code>	Static type checker and inference
<code>components/interpreter.py</code>	Tree-walking interpreter
<code>components/pipeline.py</code>	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
<code>dataclasses</code>	AST and runtime helper structures
<code>tkinter</code>	Desktop graphical interface
<code>unittest</code>	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);           (* 42 : Integer      *)
2 print(3.14);         (* 3.14 : Float      *)
3 print(true);         (* true : Boolean    *)
4 print("plc");        (* "plc" : String   *)
```

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- if execution,
- while execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *AT70.07 — Programming Language Project* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready*, a loaded filename, or *Run failed*.

Left panel (source editor). A monospaced `Text` widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A `Notebook` widget presents four tabs, each backed by a scrollable read-only `Text` widget with both horizontal and vertical scrollbars:

- *Execution Output* — the `print()` values produced by stage 4, formatted as `value : Type`;
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *AST* — the indented tree text from stage 2;

- *Type Table* — the symbol table from stage 3 showing each name, kind, inferred type, initialisation state, and parameter list.

Error handling. If any pipeline stage raises an exception, the error message is written to the *Execution Output* tab as ERROR: <message>. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

Keyboard shortcuts. F5 and Ctrl+Enter are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Test Cases

Testing was performed through both manual execution and automated regression tests. Manual runs were used to inspect the full pipeline from source code to output, while automated tests were added for the semantic and runtime behaviour of the implemented type checker and interpreter.

Table 7.1. Representative Test Cases

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate mixed numeric expressions	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```

1 IDENTIFIER  'x'      line=1 col=1
2 ASSIGN      '='      line=1 col=3
3 INT_LITERAL '10'     line=1 col=5
4 SEMICOLON   ';'      line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 *Parser and AST*

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the `IDENTIFIER+ASSIGN` lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError`: messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested using both integer-only and mixed integer-float programs. These tests verified precedence and numeric promotion. Expressions such as `x + y * 2` confirmed that multiplication is applied before addition.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker enforces that both operands of `==` and `!=` are arithmetic (`Integer` or `Float`); supplying a `Boolean` or `String` operand raises a `TypeCheckError`. The interpreter used the resulting `Boolean` values for control flow.

7.1.5 Assignment Statements

Assignments were tested for both first-use inference and reassignment checks. For example, a variable assigned 5 and later assigned "hello" correctly triggers a type error.

7.1.6 If-Then-Else

Conditional programs were used to verify that exactly one branch is executed and that non-Boolean conditions are rejected before runtime.

7.1.7 While-Loop

While-loops were tested with a countdown example. This confirmed that the loop body executes repeatedly and that `print()` emits output progressively on each iteration rather than only once at the end.

7.1.8 Functions

Function tests covered parameter passing, local execution, and return values. The implementation infers parameter types from the first call and then enforces that signature for later calls.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals ($x = -5$), float literals ($y = -3.14$), negated variable references ($y = -x$), and negation inside a larger expression ($x = 3 + -2$). In each case the type checker assigned the correct type and the interpreter produced the expected value.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.2. Automated test coverage by class

Test class	Stage covered	Tests
LexerTests	Tokenisation (<code>lexica.py</code>)	10
SymbolTableTests	Scoped symbol table (<code>symbol_table.py</code>)	7
ParserTests	Parsing and precedence (<code>parser.py</code>)	5
TypeCheckerTests	Static type inference (<code>type_checker.py</code>)	15
IntegrationTests	Full pipeline execution	17
Total		54

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identi-

fiers, two-character operators) is produced correctly, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 15 tests cover integer and float arithmetic, division producing `Float`, numeric promotion, string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types, both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
python3 -m unittest discover -s tests -v
```

All 54 tests pass.

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a working statically-typed programming language with the required primitive types: Integer, Float, Boolean, and String. The language supports arithmetic expressions (including unary negation), equality and inequality checks, assignments, if-then-else statements, while-loops, function abstraction with value parameter passing, return statements, and the built-in `print()` function.

The final system follows a clean compiler-style architecture. The lexer creates tokens, the parser builds the AST, the type checker performs static inference and validation, and the interpreter executes the program. This staged design made it easier to isolate syntax errors, type errors, and runtime behaviour.

One of the main outcomes of the project is the successful implementation of static typing without explicit declarations. Variable types are inferred from assignments, and function signatures are inferred from calls and return statements. Invalid programs are rejected before runtime. In addition, the system includes both a command-line runner and a desktop UI so that scripts can be entered, loaded, executed, and inspected conveniently.

The 54 automated tests cover all five test classes — lexical analysis, symbol table, parsing, static type checking, and full pipeline execution — and confirm correct behaviour for all language features including scope isolation, duplicate symbol detection, unary negation, division type rules, and error detection. These checks provide a baseline for future extensions.

One design consideration in the type checker is that function bodies are checked on demand. For functions that are called, the body is checked at the first call site using the inferred argument types. For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed: it infers parameter types from how they are used inside the body (for example, a parameter used in an arithmetic expression is inferred as Integer or Float), then runs the full body check with those inferred types. This ensures that type errors in never-called function bodies are still detected before execution.

Future work could include recursive functions, richer data structures, additional operators, and

code generation to a lower-level target. The current implementation demonstrates all core compiler-construction concepts covered in the course — lexical analysis, parsing, static type inference, and tree-walking interpretation — in a complete, tested, and working system.

REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

The project can be run from the command line or the desktop UI:

```
1 python3 main.py
2 python3 ui.py
3 python3 -m unittest discover -s tests -v
```

No external Python dependencies are required.

A.1 Entry Points

```
1 from __future__ import annotations
2
3 import argparse
4 from pathlib import Path
5
6 from components.pipeline import format_stage_output, run_pipeline
7
8
9 DEFAULT_SOURCE = """
10 def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }
15
16 x = 10;
17 y = 20.5;
18 name = "plc";
19 flag = true;
20
21 if (x != 0) {
22     print(name);
23 } else {
24     print(x);
25 }
26
27 i = 3;
28 while (i != 0) {
29     print(i);
30     i = i - 1;
31 }
```

```

32
33 z = add(x, y);
34 print(z);
35 """
36
37
38 def run_source(source_code: str) -> None:
39     result = run_pipeline(source_code)
40     print(format_stage_output(result))
41
42
43 def _parse_args() -> argparse.Namespace:
44     parser = argparse.ArgumentParser(description="Run the programming
45         language pipeline.")
46     parser.add_argument("script", nargs="?", help="Optional path to a
47         source file to run")
48     return parser.parse_args()
49
50 def main() -> None:
51     args = _parse_args()
52     if args.script:
53         source_code = Path(args.script).read_text(encoding="utf-8")
54     else:
55         source_code = DEFAULT_SOURCE
56     run_source(source_code)
57
58 if __name__ == "__main__":
59     main()

```

Listing 8.1: main.py — command-line entry point

```

1 from __future__ import annotations
2
3 import tkinter as tk
4 from pathlib import Path
5 from tkinter import filedialog, ttk
6
7 from components.pipeline import format_tokens, run_pipeline
8
9
10 DEFAULT_SOURCE = """def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }
15
16 x = 10;
17 y = 20.5;
18 name = \"plc\";
19 flag = true;
20

```

```

21 if (x != 0) {
22     print(name);
23 } else {
24     print(x);
25 }
26
27 i = 3;
28 while (i != 0) {
29     print(i);
30     i = i - 1;
31 }
32
33 z = add(x, y);
34 print(z);
35 """
36
37
38 class LanguageWorkbench:
39     def __init__(self) -> None:
40         self.root = tk.Tk()
41         self.root.title("AT70.07 -- Programming Language Project")
42         self.root.geometry("1200x760")
43         self.current_file: Path | None = None
44
45         self._build_layout()
46         self.source_text.insert("1.0", DEFAULT_SOURCE)
47         self.root.bind("<F5>", lambda _e: self._run_source())
48         self.root.bind("<Control-Return>", lambda _e: self._run_source
49             ())
50
51     def _build_layout(self) -> None:
52         toolbar = ttk.Frame(self.root, padding=10)
53         toolbar.pack(fill=tk.X)
54
55         ttk.Button(toolbar, text="Open Script", command=self.
56             _open_script).pack(side=tk.LEFT)
57         ttk.Button(toolbar, text="Run", command=self._run_source).pack(
58             side=tk.LEFT, padx=(8, 0))
59         ttk.Button(toolbar, text="Clear Output", command=self.
60             _clear_output).pack(side=tk.LEFT, padx=(8, 0))
61
62         self.status_var = tk.StringVar(value="Ready")
63         ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.
64             RIGHT)
65
66         body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
67         body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
68
69         left_panel = ttk.Frame(body, padding=8)
70         right_panel = ttk.Frame(body, padding=8)
71         body.add(left_panel, weight=1)
72         body.add(right_panel, weight=1)

```

```

69     ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
70     src_frame = ttk.Frame(left_panel)
71     src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
72     self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=(
73         "Consolas", 11))
74     src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=
75         self.source_text.yview)
76     src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL,
77         command=self.source_text.xview)
78     self.source_text.configure(yscrollcommand=src_vsb.set,
79         xscrollcommand=src_hsb.set)
80     src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
81     src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
82     self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
83
84     notebook = ttk.Notebook(right_panel)
85     notebook.pack(fill=tk.BOTH, expand=True)
86
87     self.output_text = self._make_output_tab(notebook, "Execution
88         Output")
89     self.tokens_text = self._make_output_tab(notebook, "Tokens")
90     self.ast_text = self._make_output_tab(notebook, "AST")
91     self.types_text = self._make_output_tab(notebook, "Type Table")
92
93     def _make_output_tab(self, notebook: ttk.Notebook, title: str) ->
94         tk.Text:
95         frame = ttk.Frame(notebook, padding=8)
96         notebook.add(frame, text=title)
97         text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11),
98             state=tk.DISABLED)
99         vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.
100             yview)
101         hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.
102             xview)
103         text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
104         vsb.pack(side=tk.RIGHT, fill=tk.Y)
105         hsb.pack(side=tk.BOTTOM, fill=tk.X)
106         text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
107         return text
108
109     def _open_script(self) -> None:
110         file_path = filedialog.askopenfilename(
111             title="Open Script File",
112             filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files"
113                 , ".*")],
114         )
115         if not file_path:
116             return
117
118         path = Path(file_path)
119         self.current_file = path
120         self.source_text.delete("1.0", tk.END)
121         self.source_text.insert("1.0", path.read_text(encoding="utf-8"))

```

```

112         )
113         self.status_var.set(f"Loaded {path.name}")
114
115     def _run_source(self) -> None:
116         source = self.source_text.get("1.0", tk.END)
117         try:
118             result = run_pipeline(source)
119         except Exception as error:
120             self._write_text(self.output_text, f"ERROR: {error}")
121             self.status_var.set("Run failed")
122             return
123
124         self._write_text(self.output_text, "\n".join(result.outputs) if
125             result.outputs else "(no output)")
126         self._write_text(self.tokens_text, format_tokens(result.tokens)
127             )
128         self._write_text(self.ast_text, result.ast_text)
129         self._write_text(self.types_text, result.checked_scope.
130             format_table())
131
132         current_label = self.current_file.name if self.current_file is
133             not None else "editor contents"
134         self.status_var.set(f"Ran {current_label}")
135
136     def _clear_output(self) -> None:
137         self._write_text(self.output_text, "")
138         self._write_text(self.tokens_text, "")
139         self._write_text(self.ast_text, "")
140         self._write_text(self.types_text, "")
141         self.status_var.set("Output cleared")
142
143     @staticmethod
144     def _write_text(widget: tk.Text, content: str) -> None:
145         widget.config(state=tk.NORMAL)
146         widget.delete("1.0", tk.END)
147         widget.insert("1.0", content)
148         widget.config(state=tk.DISABLED)
149
150     def run(self) -> None:
151         self.root.mainloop()
152
153 if __name__ == "__main__":
154     LanguageWorkbench().run()

```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass

```

```

4 from enum import Enum, auto
5
6
7 class TokenType(Enum):
8     # Literals
9     INT_LITERAL = auto()
10    FLOAT_LITERAL = auto()
11    BOOL_LITERAL = auto()
12    STRING_LITERAL = auto()
13
14    # Identifier
15    IDENTIFIER = auto()
16
17    # Keywords
18    IF = auto()
19    ELSE = auto()
20    WHILE = auto()
21    DEF = auto()
22    RETURN = auto()
23    PRINT = auto()
24
25    # Operators
26    PLUS = auto()
27    MINUS = auto()
28    STAR = auto()
29    SLASH = auto()
30    EQUAL_EQUAL = auto()
31    BANG_EQUAL = auto()
32    ASSIGN = auto()
33
34    # Delimiters
35    LPAREN = auto()
36    RPAREN = auto()
37    LBRACE = auto()
38    RBRACE = auto()
39    COMMA = auto()
40    SEMICOLON = auto()
41
42    EOF = auto()
43
44
45 @dataclass(frozen=True)
46 class Token:
47     token_type: TokenType
48     lexeme: str
49     line: int
50     column: int

```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```

1 from __future__ import annotations
2
3 from typing import Iterator

```

```

4
5 from components.tokens import Token, TokenType
6
7
8 KEYWORDS: dict[str, TokenType] = {
9     "if": TokenType.IF,
10    "else": TokenType.ELSE,
11    "while": TokenType.WHILE,
12    "def": TokenType.DEF,
13    "return": TokenType.RETURN,
14    "print": TokenType.PRINT,
15    "true": TokenType.BOOL_LITERAL,
16    "false": TokenType.BOOL_LITERAL,
17 }
18
19
20 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
21     "+": TokenType.PLUS,
22     "-": TokenType.MINUS,
23     "*": TokenType.STAR,
24     "/": TokenType.SLASH,
25     "=": TokenType.ASSIGN,
26     "(": TokenType.LPAREN,
27     ")": TokenType.RPAREN,
28     "{": TokenType.LBRACE,
29     "}": TokenType.RBRACE,
30     ",": TokenType.COMMA,
31     ";": TokenType.SEMICOLON,
32 }
33
34
35 class LexerError(Exception):
36     pass
37
38
39 class Lexer:
40     def __init__(self, source: str) -> None:
41         self.source = source
42         self.start = 0
43         self.current = 0
44         self.line = 1
45         self.column = 1
46         self.token_column = 1
47
48     def tokenize(self) -> list[Token]:
49         tokens: list[Token] = []
50         while not self._is_at_end():
51             self.start = self.current
52             self.token_column = self.column
53             token = self._scan_token()
54             if token is not None:
55                 tokens.append(token)
56         tokens.append(Token(TokenType.EOF, "", self.line, self.column))

```

```

57         return tokens
58
59     def _scan_token(self) -> Token | None:
60         char = self._advance()
61
62         # Whitespace handling
63         if char in {" ", "\t", "\r"}:
64             return None
65         if char == "\n":
66             self.line += 1
67             self.column = 1
68             return None
69
70         # Two-char operators
71         if char == "=":
72             if self._match("="):
73                 return self._make_token(TokenType.EQUAL_EQUAL)
74             return self._make_token(TokenType.ASSIGN)
75         if char == "!":
76             if self._match("="):
77                 return self._make_token(TokenType.BANG_EQUAL)
78             raise LexerError(self._error_message("Unexpected '!' . Did
              you mean '!='?"))
79
80         # Single-char operators/delimiters
81         single_char_token = SINGLE_CHAR_TOKENS.get(char)
82         if single_char_token is not None:
83             return self._make_token(single_char_token)
84
85         if char == "'":
86             return self._string()
87
88         if char.isdigit():
89             return self._number()
90
91         if self._is_identifier_start(char):
92             return self._identifier()
93
94         raise LexerError(self._error_message(f"Unexpected character: {
              char!r}"))
95
96     def _string(self) -> Token:
97         while self._peek() != "'" and not self._is_at_end():
98             if self._peek() == "\n":
99                 self.line += 1
100                 self.column = 1
101                 self._advance()
102
103         if self._is_at_end():
104             raise LexerError(self._error_message("Unterminated string
              literal"))
105
106         # Closing quote

```

```

107         self._advance()
108         return self._make_token(TokenType.STRING_LITERAL)
109
110     def _number(self) -> Token:
111         while self._peek().isdigit():
112             self._advance()
113
114         is_float = False
115         if self._peek() == "." and self._peek_next().isdigit():
116             is_float = True
117             self._advance()
118             while self._peek().isdigit():
119                 self._advance()
120
121         if is_float:
122             return self._make_token(TokenType.FLOAT_LITERAL)
123         return self._make_token(TokenType.INT_LITERAL)
124
125     def _identifier(self) -> Token:
126         while self._is_identifier_part(self._peek()):
127             self._advance()
128
129         lexeme = self._current_lexeme()
130         token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
131         return self._make_token(token_type)
132
133     def _current_lexeme(self) -> str:
134         return self.source[self.start : self.current]
135
136     def _make_token(self, token_type: TokenType) -> Token:
137         return Token(token_type, self._current_lexeme(), self.line,
138                     self.token_column)
139
140     def _advance(self) -> str:
141         char = self.source[self.current]
142         self.current += 1
143         self.column += 1
144         return char
145
146     def _match(self, expected: str) -> bool:
147         if self._is_at_end():
148             return False
149         if self.source[self.current] != expected:
150             return False
151
152         self.current += 1
153         self.column += 1
154         return True
155
156     def _peek(self) -> str:
157         if self._is_at_end():
158             return "\0"
159         return self.source[self.current]

```

```

159
160     def _peek_next(self) -> str:
161         if self.current + 1 >= len(self.source):
162             return "\0"
163         return self.source[self.current + 1]
164
165     def _is_at_end(self) -> bool:
166         return self.current >= len(self.source)
167
168     @staticmethod
169     def _is_identifier_start(char: str) -> bool:
170         return char.isalpha() or char == "_"
171
172     @staticmethod
173     def _is_identifier_part(char: str) -> bool:
174         return char.isalnum() or char == "_"
175
176     def _error_message(self, message: str) -> str:
177         return f"[line {self.line}, col {self.token_column}] {message}"
178
179
180 def iter_tokens(source: str) -> Iterator[Token]:
181     yield from Lexer(source).tokenize()

```

Listing 8.4: components/lexica.py — lexical analyser

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from typing import Optional
5
6
7 #
8
9 # Base
10
11 @dataclass
12 class ASTNode:
13     """Base class for every AST node. Carries source location for
14     diagnostics."""
15     line: int = field(default=0, kw_only=True)
16     column: int = field(default=0, kw_only=True)
17
18 #
19
20 # Expressions
21
22 #

```

```

21
22 @dataclass
23 class Literal(ASTNode):
24     """An integer, float, boolean, or string literal.
25
26     Examples:  42    3.14    true    "hello"
27     """
28     value: int | float | bool | str
29
30
31 @dataclass
32 class Identifier(ASTNode):
33     """A variable or function name used as an expression.
34
35     Example:  x
36     """
37     name: str
38
39
40 @dataclass
41 class BinaryOp(ASTNode):
42     """An arithmetic or comparison binary operation.
43
44     op is one of: '+' '-' '*' '/' '==' '!='
45     Example:  a + b    x == 0
46     """
47     op: str
48     left: ASTNode
49     right: ASTNode
50
51
52 @dataclass
53 class FunctionCall(ASTNode):
54     """A call to a user-defined function.
55
56     Example:  add(1, 2)
57     """
58     name: str
59     args: list[ASTNode] = field(default_factory=list)
60
61
62 #
63
64 # Statements
65
66
67 @dataclass
68 class Assign(ASTNode):

```

```

68     """Variable assignment (first use infers type; later uses must
69         match).
70
71     Example:  x = 10;
72     """
73     name: str
74     value: ASTNode
75
76 @dataclass
77 class Print(ASTNode):
78     """Built-in print statement.
79
80     Example:  print(x);
81     """
82     expr: ASTNode
83
84
85 @dataclass
86 class Return(ASTNode):
87     """Return statement inside a function body.
88
89     Example:  return result;
90     """
91     expr: ASTNode
92
93
94 @dataclass
95 class Block(ASTNode):
96     """A brace-enclosed sequence of statements.
97
98     Example:  { x = 1; print(x); }
99     """
100     statements: list[ASTNode] = field(default_factory=list)
101
102
103 @dataclass
104 class If(ASTNode):
105     """If / optional else control flow.
106
107     Example:  if (x != 0) { print(x); } else { print(0); }
108     """
109     condition: ASTNode
110     then_block: Block
111     else_block: Optional[Block]
112
113
114 @dataclass
115 class While(ASTNode):
116     """While loop.
117
118     Example:  while (x != 0) { x = x - 1; }
119     """

```

```

120     condition: ASTNode
121     body: Block
122
123
124 @dataclass
125 class FunctionDef(ASTNode):
126     """Function definition.
127
128     Example:  def add(a, b) { return a + b; }
129
130     Note: parameter types are unknown at parse time -- Member 3's type
131           checker
132           will infer them from usage. We store only the parameter names here.
133     """
134     name: str
135     params: list[str]
136     body: Block
137
138 #
139 # -----
140 # Top-level
141 # -----
142
143 @dataclass
144 class Program(ASTNode):
145     """Root node: the entire source file as a list of statements."""
146     statements: list[ASTNode] = field(default_factory=list)

```

Listing 8.5: components/ast_nodes.py — AST node definitions

```

1 from __future__ import annotations
2
3 from typing import Optional
4
5 from components.tokens import Token, TokenType
6 from components.ast_nodes import (
7     ASTNode,
8     Program,
9     Block,
10    Assign,
11    If,
12    While,
13    FunctionDef,
14    FunctionCall,
15    Print,
16    Return,
17    BinaryOp,
18    Literal,
19    Identifier,

```

```

20 )
21
22
23 #
24     -----
25
26
27 # Parser error
28 #
29     -----
30
31
32 class ParseError(Exception):
33     pass
34
35 #
36     -----
37
38 # Parser
39 #
40     -----
41
42
43 class Parser:
44     """Recursive-descent parser.
45
46     Usage:
47         tokens = Lexer(source).tokenize()
48         tree   = Parser(tokens).parse()    # returns Program node
49     """
50
51     def __init__(self, tokens: list[Token]) -> None:
52         self._tokens = tokens
53         self._pos = 0
54
55     #
56         -----
57
58     # Public entry point
59     #
60         -----
61
62
63     def parse(self) -> Program:
64         """Parse the full token stream and return the root Program node
65         ."""
66
67         line, col = self._peek().line, self._peek().column
68         statements: list[ASTNode] = []
69         while not self._is_at_end():
70             statements.append(self._statement())
71         return Program(statements=statements, line=line, column=col)
72
73     #

```

```

60  # Statements
61  #
62
63  def _statement(self) -> ASTNode:
64      """Dispatch to the correct statement parser based on the
65         current token."""
66      tok = self._peek()
67
68      if tok.token_type == TokenType.DEF:
69          return self._func_def()
70
71      if tok.token_type == TokenType.IF:
72          return self._if_stmt()
73
74      if tok.token_type == TokenType.WHILE:
75          return self._while_stmt()
76
77      if tok.token_type == TokenType.RETURN:
78          return self._return_stmt()
79
80      if tok.token_type == TokenType.PRINT:
81          return self._print_stmt()
82
83      # assignment: IDENTIFIER "=" expr ";"
84      if (
85          tok.token_type == TokenType.IDENTIFIER
86          and self._peek_next().token_type == TokenType.ASSIGN
87      ):
88          return self._assignment()
89
90      # fallback: bare expression statement expr ";"
91      node = self._expr()
92      self._expect(TokenType.SEMICOLON, "Expected ';' after
93         expression")
94      return node
95
96  def _assignment(self) -> Assign:
97      """IDENTIFIER '=' expr ';'"""
98      name_tok = self._advance() #
99      IDENTIFIER # '='
100      self._advance()
101      value = self._expr()
102      self._expect(TokenType.SEMICOLON, "Expected ';' after
103         assignment")
104      return Assign(name=name_tok.lexeme, value=value,
105                    line=name_tok.line, column=name_tok.column)
106
107  def _if_stmt(self) -> If:
108      """'if' '(' bool_expr ')' block ( 'else' block )?"""

```

```

105     tok = self._advance()                                     # 'if'
106     self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
107     condition = self._bool_expr()
108     self._expect(TokenType.RPAREN, "Expected ')' after if condition
109         ")
110     then_block = self._block()
111     else_block: Optional[Block] = None
112     if self._check(TokenType.ELSE):
113         self._advance()                                       # 'else'
114         else_block = self._block()
115     return If(condition=condition, then_block=then_block,
116         else_block=else_block,
117         line=tok.line, column=tok.column)
118
119 def _while_stmt(self) -> While:
120     """'while' '(' bool_expr ')' block"""
121     tok = self._advance()                                     # '
122     while'
123     self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
124     condition = self._bool_expr()
125     self._expect(TokenType.RPAREN, "Expected ')' after while
126         condition")
127     body = self._block()
128     return While(condition=condition, body=body,
129         line=tok.line, column=tok.column)
130
131 def _func_def(self) -> FunctionDef:
132     """'def' IDENTIFIER '(' params? ')' block"""
133     tok = self._advance()                                     # 'def'
134     name_tok = self._expect(TokenType.IDENTIFIER, "Expected
135         function name after 'def'")
136     self._expect(TokenType.LPAREN, "Expected '(' after function
137         name")
138     params = self._params()
139     self._expect(TokenType.RPAREN, "Expected ')' after parameters")
140     body = self._block()
141     return FunctionDef(name=name_tok.lexeme, params=params, body=
142         body,
143         line=tok.line, column=tok.column)
144
145 def _print_stmt(self) -> Print:
146     """'print' '(' expr ')' ';'"""
147     tok = self._advance()                                     # '
148     print'
149     self._expect(TokenType.LPAREN, "Expected '(' after 'print'")
150     expr = self._expr()
151     self._expect(TokenType.RPAREN, "Expected ')' after print
152         expression")
153     self._expect(TokenType.SEMICOLON, "Expected ';' after print
154         statement")
155     return Print(expr=expr, line=tok.line, column=tok.column)

```

```

147 def _return_stmt(self) -> Return:
148     """'return' expr ';'"""
149     tok = self._advance() # '
150     return '
151     expr = self._expr()
152     self._expect(TokenType.SEMICOLON, "Expected ';' after return
153         value")
154     return Return(expr=expr, line=tok.line, column=tok.column)
155
156 def _block(self) -> Block:
157     """'{', statement* '}'"""
158     tok = self._expect(TokenType.LBRACE, "Expected '{'")
159     statements: list[ASTNode] = []
160     while not self._check(TokenType.RBRACE) and not self._is_at_end
161         ():
162         statements.append(self._statement())
163     self._expect(TokenType.RBRACE, "Expected '}' to close block")
164     return Block(statements=statements, line=tok.line, column=tok.
165         column)
166
167 #
168 -----
169
170 # Parameters (function definition)
171 #
172 -----
173
174 def _params(self) -> list[str]:
175     """IDENTIFIER (',' IDENTIFIER)* -- returns list of param names.
176     """
177     params: list[str] = []
178     if self._check(TokenType.IDENTIFIER):
179         params.append(self._advance().lexeme)
180         while self._check(TokenType.COMMA):
181             self._advance() # ',',
182             tok = self._expect(TokenType.IDENTIFIER, "Expected
183                 parameter name after ','")
184             params.append(tok.lexeme)
185     return params
186
187 #
188 -----
189
190 # Boolean expressions (lowest precedence, used in if/while
191     conditions)
192 #
193 -----
194
195 def _bool_expr(self) -> ASTNode:
196     """expr ('==' | '!=') expr | BOOL_LITERAL"""
197     # bare boolean literal: true / false

```

```

185     if self._check(TokenType.BOOL_LITERAL):
186         tok = self._advance()
187         value = tok.lexeme == "true"
188         return Literal(value=value, line=tok.line, column=tok.
189             column)
190
191     left = self._expr()
192
193     if self._check(TokenType.EQUAL_EQUAL):
194         op_tok = self._advance()
195         right = self._expr()
196         return BinaryOp(op=="==", left=left, right=right,
197             line=op_tok.line, column=op_tok.column)
198
199     if self._check(TokenType.BANG_EQUAL):
200         op_tok = self._advance()
201         right = self._expr()
202         return BinaryOp(op="!=", left=left, right=right,
203             line=op_tok.line, column=op_tok.column)
204
205     # no comparison operator found -- return the expr as-is
206     # (the type checker will validate it's Boolean)
207     return left
208
209     #
210     -----
211
212     # Arithmetic expressions (standard precedence)
213     #
214     -----
215
216     def _expr(self) -> ASTNode:
217         """term (('+' | '-') term)"""
218         node = self._term()
219         while self._check(TokenType.PLUS) or self._check(TokenType.
220             MINUS):
221             op_tok = self._advance()
222             right = self._term()
223             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
224                 line=op_tok.line, column=op_tok.column)
225         return node
226
227     def _term(self) -> ASTNode:
228         """factor (('*' | '/') factor)"""
229         node = self._factor()
230         while self._check(TokenType.STAR) or self._check(TokenType.
231             SLASH):
232             op_tok = self._advance()
233             right = self._factor()
234             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
235                 line=op_tok.line, column=op_tok.column)
236         return node

```

```

231
232 def _factor(self) -> ASTNode:
233     """
234     '-' factor (unary negation, desugared to 0 - factor)
235     | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
236     | IDENTIFIER ( '(' args? ')' )?
237     | '(' expr ')'
238     """
239     tok = self._peek()
240
241     # Unary minus '-' factor
242     if tok.token_type == TokenType.MINUS:
243         op_tok = self._advance()
244         operand = self._factor()
245         zero = Literal(value=0, line=op_tok.line, column=op_tok.
246                        column)
247         return BinaryOp(op="-", left=zero, right=operand,
248                        line=op_tok.line, column=op_tok.column)
249
250     # Integer literal
251     if tok.token_type == TokenType.INT_LITERAL:
252         self._advance()
253         return Literal(value=int(tok.lexeme), line=tok.line, column
254                        =tok.column)
255
256     # Float literal
257     if tok.token_type == TokenType.FLOAT_LITERAL:
258         self._advance()
259         return Literal(value=float(tok.lexeme), line=tok.line,
260                        column=tok.column)
261
262     # Boolean literal
263     if tok.token_type == TokenType.BOOL_LITERAL:
264         self._advance()
265         return Literal(value=(tok.lexeme == "true"), line=tok.line,
266                        column=tok.column)
267
268     # String literal (keep the surrounding quotes stripped)
269     if tok.token_type == TokenType.STRING_LITERAL:
270         self._advance()
271         return Literal(value=tok.lexeme[1:-1], line=tok.line,
272                        column=tok.column)
273
274     # Identifier -- could be a plain variable or a function call
275     if tok.token_type == TokenType.IDENTIFIER:
276         self._advance()
277         if self._check(TokenType.LPAREN):
278             # function call
279             self._advance() # '('
280             args = self._args()
281             self._expect(TokenType.RPAREN, "Expected ')' after
282                          function arguments")
283             return FunctionCall(name=tok.lexeme, args=args,

```

```

278         line=tok.line, column=tok.column)
279     return Identifier(name=tok.lexeme, line=tok.line, column=
        tok.column)
280
281     # Grouped expression '(' expr ')'
282     if tok.token_type == TokenType.LPAREN:
283         self._advance()                                # '('
284         node = self._expr()
285         self._expect(TokenType.RPAREN, "Expected ')' to close
            grouped expression")
286         return node
287
288     raise ParseError(self._error_at(tok, f"Unexpected token: {tok.
        lexeme!r}"))
289
290     #
291     -----
292
293     # Arguments (function call)
294     #
295     -----
296
297     def _args(self) -> list[ASTNode]:
298         """expr (',' expr)*"""
299         args: list[ASTNode] = []
300         if not self._check(TokenType.RPAREN):
301             args.append(self._expr())
302             while self._check(TokenType.COMMA):
303                 self._advance()                                # ','
304                 args.append(self._expr())
305             return args
306
307         #
308         -----
309
310     # Token navigation helpers
311     #
312     -----
313
314     def _peek(self) -> Token:
315         return self._tokens[self._pos]
316
317     def _peek_next(self) -> Token:
318         if self._pos + 1 < len(self._tokens):
319             return self._tokens[self._pos + 1]
320         return self._tokens[-1]                                # EOF
321
322     def _advance(self) -> Token:
323         tok = self._tokens[self._pos]
324         if not self._is_at_end():
325             self._pos += 1

```

```

320         return tok
321
322     def _check(self, token_type: TokenType) -> bool:
323         return self._peek().token_type == token_type
324
325     def _is_at_end(self) -> bool:
326         return self._peek().token_type == TokenType.EOF
327
328     def _expect(self, token_type: TokenType, message: str) -> Token:
329         if self._check(token_type):
330             return self._advance()
331         raise ParseError(self._error_at(self._peek(), message))
332
333     def _error_at(self, tok: Token, message: str) -> str:
334         return f"[line {tok.line}, col {tok.column}] ParseError: {
            message}"

```

Listing 8.6: components/parser.py — recursive-descent parser

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass, field
4  from enum import Enum
5
6
7  class LanguageType(Enum):
8      INTEGER = "Integer"
9      FLOAT = "Float"
10     BOOLEAN = "Boolean"
11     STRING = "String"
12     VOID = "Void"
13
14
15     @dataclass
16     class Symbol:
17         name: str
18         symbol_type: LanguageType
19
20
21     @dataclass
22     class VariableSymbol(Symbol):
23         initialized: bool = False
24
25
26     @dataclass
27     class FunctionSymbol(Symbol):
28         parameters: list[tuple[str, LanguageType]] = field(default_factory=
            list)
29
30
31     class SymbolTableError(Exception):
32         pass
33

```

```

34
35 class SymbolTable:
36     def __init__(self, parent: SymbolTable | None = None) -> None:
37         self.parent = parent
38         self._symbols: dict[str, Symbol] = {}
39
40     def define_variable(self, name: str, symbol_type: LanguageType,
41         initialized: bool = False) -> VariableSymbol:
42         if name in self._symbols:
43             raise SymbolTableError(f"Symbol already defined in this
44                 scope: {name}")
45         symbol = VariableSymbol(name=name, symbol_type=symbol_type,
46             initialized=initialized)
47         self._symbols[name] = symbol
48         return symbol
49
50     def define_function(
51         self,
52         name: str,
53         return_type: LanguageType,
54         parameters: list[tuple[str, LanguageType]],
55     ) -> FunctionSymbol:
56         if name in self._symbols:
57             raise SymbolTableError(f"Symbol already defined in this
58                 scope: {name}")
59         symbol = FunctionSymbol(name=name, symbol_type=return_type,
60             parameters=parameters)
61         self._symbols[name] = symbol
62         return symbol
63
64     def lookup(self, name: str) -> Symbol:
65         symbol = self._symbols.get(name)
66         if symbol is not None:
67             return symbol
68         if self.parent is not None:
69             return self.parent.lookup(name)
70         raise SymbolTableError(f"Undefined symbol: {name}")
71
72     def exists_in_current_scope(self, name: str) -> bool:
73         return name in self._symbols
74
75     def child_scope(self) -> SymbolTable:
76         return SymbolTable(parent=self)
77
78     def set_initialized(self, name: str) -> None:
79         symbol = self.lookup(name)
80         if not isinstance(symbol, VariableSymbol):
81             raise SymbolTableError(f"Cannot initialize non-variable
82                 symbol: {name}")
83         symbol.initialized = True
84
85     def snapshot(self) -> dict[str, Symbol]:
86         return dict(self._symbols)

```

```

81
82 def format_table(self) -> str:
83     lines = [
84         f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized'
85             ':<12} Parameters",
86         "-" * 72,
87     ]
88
89     for name, symbol in self._symbols.items():
90         if isinstance(symbol, FunctionSymbol):
91             params = ", ".join(f"{param_name}: {param_type.value}"
92                 for param_name, param_type in symbol.parameters)
93             lines.append(
94                 f"{name:<12} {'function':<10} {symbol.symbol_type.
95                     value:<10} {'-':<12} {params or '-'}"
96             )
97         elif isinstance(symbol, VariableSymbol):
98             initialized = "yes" if symbol.initialized else "no"
99             lines.append(
100                 f"{name:<12} {'variable':<10} {symbol.symbol_type.
101                     value:<10} {initialized:<12} -"
102             )
103         else:
104             lines.append(f"{name:<12} {'symbol':<10} {symbol.
105                 symbol_type.value:<10} {'-':<12} -")
106
107     return "\n".join(lines)

```

Listing 8.7: components/symbol_table.py — scoped symbol table

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from components.ast_nodes import (
6     ASTNode,
7     Assign,
8     BinaryOp,
9     Block,
10    FunctionCall,
11    FunctionDef,
12    Identifier,
13    If,
14    Literal,
15    Print,
16    Program,
17    Return,
18    While,
19 )
20 from components.symbol_table import FunctionSymbol, LanguageType,
21     SymbolTable, SymbolTableError, VariableSymbol
22

```

```

23 class TypeCheckError(Exception):
24     pass
25
26
27 @dataclass
28 class _FunctionState:
29     node: FunctionDef
30     body_checked: bool = False
31
32
33 @dataclass
34 class _FunctionContext:
35     name: str
36     return_type: LanguageType | None = None
37
38
39 class TypeChecker:
40     def __init__(self) -> None:
41         self.global_scope = SymbolTable()
42         self._functions: dict[str, _FunctionState] = {}
43         self._active_calls: list[str] = []
44
45     def check(self, program: Program) -> SymbolTable:
46         for statement in program.statements:
47             if isinstance(statement, FunctionDef):
48                 self._register_function(statement)
49
50         for statement in program.statements:
51             if isinstance(statement, FunctionDef):
52                 continue
53             self._check_statement(statement, self.global_scope, None)
54
55         self._check_uncalled_functions()
56         return self.global_scope
57
58     def _register_function(self, node: FunctionDef) -> None:
59         if node.name in self._functions:
60             raise TypeCheckError(self._error_at(node, f"Function '{node
61                 .name}' is already defined"))
62
63         placeholder_parameters = [(param_name, LanguageType.VOID) for
64             param_name in node.params]
65         try:
66             self.global_scope.define_function(node.name, LanguageType.
67                 VOID, placeholder_parameters)
68         except SymbolTableError as error:
69             raise TypeCheckError(self._error_at(node, str(error))) from
70                 error
71         self._functions[node.name] = _FunctionState(node=node)
72
73     def _check_statement(
74         self,
75         node: ASTNode,

```

```

72     scope: SymbolTable,
73     function_context: _FunctionContext | None,
74 ) -> None:
75     if isinstance(node, Assign):
76         value_type = self._infer_expr_type(node.value, scope)
77         self._assign_variable_type(scope, node, value_type)
78         return
79
80     if isinstance(node, Print):
81         self._infer_expr_type(node.expr, scope)
82         return
83
84     if isinstance(node, Return):
85         if function_context is None:
86             raise TypeCheckError(self._error_at(node, "'return' is
87                 only valid inside a function"))
88         expr_type = self._infer_expr_type(node.expr, scope)
89         if function_context.return_type is None:
90             function_context.return_type = expr_type
91         elif function_context.return_type != expr_type:
92             raise TypeCheckError(
93                 self._error_at(
94                     node,
95                     f"Function '{function_context.name}' returns
96                         both {function_context.return_type.value}
97                         and {expr_type.value}",
98                 )
99             )
100         return
101
102     if isinstance(node, If):
103         condition_type = self._infer_expr_type(node.condition,
104             scope)
105         if condition_type != LanguageType.BOOLEAN:
106             raise TypeCheckError(self._error_at(node.condition, "If
107                 condition must have type Boolean"))
108         self._check_block(node.then_block, scope.child_scope(),
109             function_context)
110         if node.else_block is not None:
111             self._check_block(node.else_block, scope.child_scope(),
112                 function_context)
113         return
114
115     if isinstance(node, While):
116         condition_type = self._infer_expr_type(node.condition,
117             scope)
118         if condition_type != LanguageType.BOOLEAN:
119             raise TypeCheckError(self._error_at(node.condition, "
120                 While condition must have type Boolean"))
121         self._check_block(node.body, scope.child_scope(),
122             function_context)
123         return

```

```

115         if isinstance(node, Block):
116             self._check_block(node, scope.child_scope(),
117                               function_context)
118             return
119
120         if isinstance(node, FunctionCall):
121             self._infer_function_call_type(node, scope)
122             return
123
124         raise TypeCheckError(self._error_at(node, f"Unsupported
125             statement node: {type(node).__name__}"))
126
127     def _check_block(
128         self,
129         block: Block,
130         scope: SymbolTable,
131         function_context: _FunctionContext | None,
132     ) -> None:
133         for statement in block.statements:
134             self._check_statement(statement, scope, function_context)
135
136     def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) ->
137         LanguageType:
138         if isinstance(node, Literal):
139             return self._literal_type(node)
140
141         if isinstance(node, Identifier):
142             try:
143                 symbol = scope.lookup(node.name)
144             except SymbolTableError as error:
145                 raise TypeCheckError(self._error_at(node, str(error)))
146                 from error
147             if not isinstance(symbol, VariableSymbol):
148                 raise TypeCheckError(self._error_at(node, f" '{node.name
149                     }' is not a variable"))
150             return symbol.symbol_type
151
152         if isinstance(node, FunctionCall):
153             return self._infer_function_call_type(node, scope)
154
155         if isinstance(node, BinaryOp):
156             left_type = self._infer_expr_type(node.left, scope)
157             right_type = self._infer_expr_type(node.right, scope)
158             return self._infer_binary_type(node, left_type, right_type)
159
160         raise TypeCheckError(self._error_at(node, f"Unsupported
161             expression node: {type(node).__name__}"))
162
163     def _infer_function_call_type(self, node: FunctionCall, scope:
164         SymbolTable) -> LanguageType:
165         function_state = self._functions.get(node.name)
166         if function_state is None:
167             raise TypeCheckError(self._error_at(node, f"Undefined

```

```

        function: {node.name}"))
161 if node.name in self._active_calls:
162     raise TypeCheckError(self._error_at(node, f"Recursive
        function calls are not supported: {node.name}"))
163
164 argument_types = [self._infer_expr_type(argument, scope) for
        argument in node.args]
165 function_symbol = self._lookup_function_symbol(node)
166
167 if len(argument_types) != len(function_symbol.parameters):
168     raise TypeCheckError(
169         self._error_at(
170             node,
171             f"Function '{node.name}' expects {len(
                function_symbol.parameters)} argument(s), got {
                len(argument_types)}",
172         )
173     )
174
175 declared_parameter_types = [param_type for _, param_type in
        function_symbol.parameters]
176 if all(param_type == LanguageType.VOID for param_type in
        declared_parameter_types):
177     function_symbol.parameters = list(zip(function_state.node.
        params, argument_types, strict=False))
178 else:
179     for index, ((_, expected_type), actual_type) in enumerate(
        zip(function_symbol.parameters, argument_types, strict=
        False)):
180         if expected_type != actual_type:
181             raise TypeCheckError(
182                 self._error_at(
183                     node.args[index],
184                     f"Argument {index + 1} of '{node.name}'
                        must be {expected_type.value}, got {
                        actual_type.value}",
185                 )
186             )
187
188 if not function_state.body_checked:
189     self._check_function_body(function_state, function_symbol)
190
191 return function_symbol.symbol_type
192
193 def _check_function_body(self, function_state: _FunctionState,
        function_symbol: FunctionSymbol) -> None:
194     function_context = _FunctionContext(name=function_state.node.
        name)
195     function_scope = self.global_scope.child_scope()
196     for parameter_name, parameter_type in function_symbol.
        parameters:
197         function_scope.define_variable(parameter_name,
            parameter_type, initialized=True)

```

```

198     self._active_calls.append(function_state.node.name)
199     try:
200         self._check_block(function_state.node.body, function_scope,
201                             function_context)
202     finally:
203         self._active_calls.pop()
204
205     function_symbol.symbol_type = function_context.return_type or
206     LanguageType.VOID
207     function_state.body_checked = True
208
209 def _check_uncalled_functions(self) -> None:
210     """After all call-site checks, type-check any function whose
211     body was never visited."""
212     for name, state in self._functions.items():
213         if not state.body_checked:
214             try:
215                 function_symbol = self.global_scope.lookup(name)
216             except SymbolTableError:
217                 continue
218             if not isinstance(function_symbol, FunctionSymbol):
219                 continue
220             inferred_params = self._infer_param_types_from_body(
221                 state.node)
222             function_symbol.parameters = inferred_params
223             self._check_function_body(state, function_symbol)
224
225 def _infer_param_types_from_body(self, node: FunctionDef) -> list[
226     tuple[str, LanguageType]]:
227     """Infer parameter types by scanning the body for expression
228     contexts.
229
230     Any parameter used as an operand of an arithmetic or comparison
231     operator
232     is inferred as numeric (Float if the sibling operand is a float
233     literal,
234     Integer otherwise). Parameters whose usage gives no type hint
235     default to
236     Integer.
237     """
238     param_names = set(node.params)
239     inferred: dict[str, LanguageType] = {}
240
241 def scan_expr(n: ASTNode) -> None:
242     if isinstance(n, BinaryOp):
243         if n.op in {"+", "-", "*", "/", "=", "!="}:
244             for operand, other in ((n.left, n.right), (n.right,
245                 n.left)):
246                 if isinstance(operand, Identifier) and operand.
247                     name in param_names:
248                     if isinstance(other, Literal) and
249                         isinstance(other.value, float):

```

```

239         inferred.setdefault(operand.name,
240                               LanguageType.FLOAT)
241     else:
242         inferred.setdefault(operand.name,
243                               LanguageType.INTEGER)
244     scan_expr(n.left)
245     scan_expr(n.right)
246     elif isinstance(n, FunctionCall):
247         for arg in n.args:
248             scan_expr(arg)
249
250 def scan_stmt(n: ASTNode) -> None:
251     if isinstance(n, Assign):
252         scan_expr(n.value)
253     elif isinstance(n, Print):
254         scan_expr(n.expr)
255     elif isinstance(n, Return):
256         scan_expr(n.expr)
257     elif isinstance(n, If):
258         scan_expr(n.condition)
259         scan_block(n.then_block)
260         if n.else_block:
261             scan_block(n.else_block)
262     elif isinstance(n, While):
263         scan_expr(n.condition)
264         scan_block(n.body)
265     elif isinstance(n, Block):
266         scan_block(n)
267     elif isinstance(n, FunctionCall):
268         for arg in n.args:
269             scan_expr(arg)
270
271 def scan_block(block: Block) -> None:
272     for stmt in block.statements:
273         scan_stmt(stmt)
274
275 scan_block(node.body)
276 return [(param, inferred.get(param, LanguageType.INTEGER)) for
277         param in node.params]
278
279 def _assign_variable_type(self, scope: SymbolTable, node: Assign,
280 value_type: LanguageType) -> None:
281     try:
282         symbol = scope.lookup(node.name)
283     except SymbolTableError:
284         scope.define_variable(node.name, value_type, initialized=
285                               True)
286     return
287
288 if not isinstance(symbol, VariableSymbol):
289     raise TypeCheckError(self._error_at(node, f"'{node.name}'
290                                         is not a variable"))
291 if symbol.symbol_type != value_type:

```

```

286         raise TypeCheckError(
287             self._error_at(
288                 node,
289                 f"Cannot assign {value_type.value} to variable '{
                    node.name}' of type {symbol.symbol_type.value}",
290             )
291         )
292     symbol.initialized = True
293
294     def _infer_binary_type(
295         self,
296         node: BinaryOp,
297         left_type: LanguageType,
298         right_type: LanguageType,
299     ) -> LanguageType:
300         if node.op in {"+", "-", "*", "/"}:
301             if not self._is_numeric(left_type) or not self._is_numeric(
302                 right_type):
303                 raise TypeCheckError(self._error_at(node, f"Operator '{
                    node.op}' requires numeric operands"))
304             if node.op == "/":
305                 return LanguageType.FLOAT
306             if left_type == LanguageType.FLOAT or right_type ==
307                 LanguageType.FLOAT:
308                 return LanguageType.FLOAT
309             return LanguageType.INTEGER
310
311         if node.op in {"==", "!="}:
312             numeric_comparison = self._is_numeric(left_type) and self.
313                 _is_numeric(right_type)
314             if not numeric_comparison:
315                 raise TypeCheckError(
316                     self._error_at(
317                         node,
318                         f"Operator '{node.op}' requires two arithmetic
319                             operands",
320                     )
321                 )
322             return LanguageType.BOOLEAN
323
324         raise TypeCheckError(self._error_at(node, f"Unsupported
325             operator: {node.op}"))
326
327     def _lookup_function_symbol(self, node: FunctionCall) ->
328         FunctionSymbol:
329         try:
330             symbol = self.global_scope.lookup(node.name)
331         except SymbolTableError as error:
332             raise TypeCheckError(self._error_at(node, str(error))) from
333                 error
334         if not isinstance(symbol, FunctionSymbol):
335             raise TypeCheckError(self._error_at(node, f"'{node.name}'
336                 is not a function"))

```

```

329         return symbol
330
331     @staticmethod
332     def _is_numeric(language_type: LanguageType) -> bool:
333         return language_type in {LanguageType.INTEGER, LanguageType.
            FLOAT}
334
335     @staticmethod
336     def _literal_type(node: Literal) -> LanguageType:
337         if isinstance(node.value, bool):
338             return LanguageType.BOOLEAN
339         if isinstance(node.value, int):
340             return LanguageType.INTEGER
341         if isinstance(node.value, float):
342             return LanguageType.FLOAT
343         if isinstance(node.value, str):
344             return LanguageType.STRING
345         raise TypeError(f"Unsupported literal value: {node.value!r}
            ")
346
347     @staticmethod
348     def _error_at(node: ASTNode, message: str) -> str:
349         return f"[line {node.line}, col {node.column}] TypeError: {
            message}"

```

Listing 8.8: components/type_checker.py — static type checker with inference

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from typing import Callable
5
6  from components.ast_nodes import (
7      ASTNode,
8      Assign,
9      BinaryOp,
10     Block,
11     FunctionCall,
12     FunctionDef,
13     Identifier,
14     If,
15     Literal,
16     Print,
17     Program,
18     Return,
19     While,
20 )
21 from components.symbol_table import LanguageType
22
23
24 class InterpreterError(Exception):
25     pass
26

```

```

27
28 class _ReturnSignal(Exception):
29     def __init__(self, value: object) -> None:
30         self.value = value
31
32
33 @dataclass
34 class _FunctionRuntime:
35     node: FunctionDef
36
37
38 class _Environment:
39     def __init__(self, parent: _Environment | None = None) -> None:
40         self.parent = parent
41         self.values: dict[str, object] = {}
42
43     def define(self, name: str, value: object) -> None:
44         self.values[name] = value
45
46     def get(self, name: str) -> object:
47         if name in self.values:
48             return self.values[name]
49         if self.parent is not None:
50             return self.parent.get(name)
51         raise InterpreterError(f"Undefined variable: {name}")
52
53     def assign(self, name: str, value: object) -> None:
54         if name in self.values:
55             self.values[name] = value
56             return
57         if self.parent is not None and self.parent.contains(name):
58             self.parent.assign(name, value)
59             return
60         self.values[name] = value
61
62     def contains(self, name: str) -> bool:
63         if name in self.values:
64             return True
65         if self.parent is not None:
66             return self.parent.contains(name)
67         return False
68
69
70 class Interpreter:
71     def __init__(self, output_callback: Callable[[str], None] | None =
None) -> None:
72         self._functions: dict[str, _FunctionRuntime] = {}
73         self._output_callback = output_callback or print
74
75     def execute(self, program: Program) -> None:
76         environment = _Environment()
77         for statement in program.statements:
78             if isinstance(statement, FunctionDef):

```

```

79         self._functions[statement.name] = _FunctionRuntime(node
80             =statement)
81
82     for statement in program.statements:
83         if isinstance(statement, FunctionDef):
84             continue
85         self._execute_statement(statement, environment)
86
87     def _execute_statement(self, node: ASTNode, environment:
88         _Environment) -> None:
89         if isinstance(node, Assign):
90             value = self._evaluate(node.value, environment)
91             environment.assign(node.name, value)
92             return
93
94         if isinstance(node, Print):
95             value = self._evaluate(node.expr, environment)
96             self._output_callback(f"{self._format_value(value)} : {self
97                 ._runtime_type(value).value}")
98             return
99
100         if isinstance(node, Return):
101             raise _ReturnSignal(self._evaluate(node.expr, environment))
102
103         if isinstance(node, If):
104             condition = self._evaluate(node.condition, environment)
105             if not isinstance(condition, bool):
106                 raise InterpreterError(self._error_at(node.condition, "
107                     If condition must evaluate to Boolean"))
108             branch = node.then_block if condition else node.else_block
109             if branch is not None:
110                 self._execute_block(branch, _Environment(parent=
111                     environment))
112             return
113
114         if isinstance(node, While):
115             while True:
116                 condition = self._evaluate(node.condition, environment)
117                 if not isinstance(condition, bool):
118                     raise InterpreterError(self._error_at(node.
119                         condition, "While condition must evaluate to
120                         Boolean"))
121                 if not condition:
122                     break
123                 self._execute_block(node.body, _Environment(parent=
124                     environment))
125             return
126
127         if isinstance(node, Block):
128             self._execute_block(node, _Environment(parent=environment))
129             return
130
131         if isinstance(node, FunctionCall):

```

```

124         self._evaluate(node, environment)
125         return
126
127         raise InterpreterError(self._error_at(node, f"Unsupported
128             statement node: {type(node).__name__}"))
129
130 def _execute_block(self, block: Block, environment: _Environment)
131     -> None:
132     for statement in block.statements:
133         self._execute_statement(statement, environment)
134
135 def _evaluate(self, node: ASTNode, environment: _Environment) ->
136     object:
137     if isinstance(node, Literal):
138         return node.value
139
140     if isinstance(node, Identifier):
141         return environment.get(node.name)
142
143     if isinstance(node, BinaryOp):
144         left_value = self._evaluate(node.left, environment)
145         right_value = self._evaluate(node.right, environment)
146         return self._evaluate_binary(node, left_value, right_value)
147
148     if isinstance(node, FunctionCall):
149         return self._call_function(node, environment)
150
151     raise InterpreterError(self._error_at(node, f"Unsupported
152         expression node: {type(node).__name__}"))
153
154 def _call_function(self, node: FunctionCall, environment:
155     _Environment) -> object:
156     function_runtime = self._functions.get(node.name)
157     if function_runtime is None:
158         raise InterpreterError(self._error_at(node, f"Undefined
159             function: {node.name}"))
160
161     if len(node.args) != len(function_runtime.node.params):
162         raise InterpreterError(
163             self._error_at(
164                 node,
165                 f"Function '{node.name}' expects {len(
166                     function_runtime.node.params)} argument(s), got
167                     {len(node.args)}",
168             )
169         )
170
171     call_environment = _Environment(parent=environment)
172     argument_values = [self._evaluate(argument, environment) for
173         argument in node.args]
174     for parameter_name, argument_value in zip(function_runtime.node
175         .params, argument_values, strict=False):
176         call_environment.define(parameter_name, argument_value)

```

```

167         try:
168             self._execute_block(function_runtime.node.body,
169                                 call_environment)
170         except _ReturnSignal as signal:
171             return signal.value
172         return None
173
174     def _evaluate_binary(self, node: BinaryOp, left_value: object,
175                          right_value: object) -> object:
176         if node.op == "+":
177             return left_value + right_value
178         if node.op == "-":
179             return left_value - right_value
180         if node.op == "*":
181             return left_value * right_value
182         if node.op == "/":
183             return left_value / right_value
184         if node.op == "==":
185             return left_value == right_value
186         if node.op == "!=":
187             return left_value != right_value
188         raise InterpreterError(self._error_at(node, f"Unsupported
189                                     operator: {node.op}"))
190
191     @staticmethod
192     def _format_value(value: object) -> str:
193         if isinstance(value, str):
194             return f'"{value}"'
195         if isinstance(value, bool):
196             return "true" if value else "false"
197         return str(value)
198
199     @staticmethod
200     def _runtime_type(value: object) -> LanguageType:
201         if isinstance(value, bool):
202             return LanguageType.BOOLEAN
203         if isinstance(value, int):
204             return LanguageType.INTEGER
205         if isinstance(value, float):
206             return LanguageType.FLOAT
207         if isinstance(value, str):
208             return LanguageType.STRING
209         return LanguageType.VOID
210
211     @staticmethod
212     def _error_at(node: ASTNode, message: str) -> str:
213         return f"[line {node.line}, col {node.column}] RuntimeError: {
214                                     message}"

```

Listing 8.9: components/interpreter.py — tree-walking interpreter

```

1 from __future__ import annotations

```

```

2
3 from dataclasses import dataclass
4
5 from components.ast_printer import ASTPrinter
6 from components.interpreter import Interpreter
7 from components.lexica import Lexer
8 from components.parser import Parser
9 from components.symbol_table import SymbolTable
10 from components.tokens import Token
11 from components.type_checker import TypeChecker
12
13
14 @dataclass
15 class PipelineResult:
16     tokens: list[Token]
17     ast_text: str
18     checked_scope: SymbolTable
19     outputs: list[str]
20
21
22 def run_pipeline(source_code: str) -> PipelineResult:
23     tokens = Lexer(source_code).tokenize()
24     tree = Parser(tokens).parse()
25     ast_text = ASTPrinter().print(tree)
26     checked_scope = TypeChecker().check(tree)
27
28     outputs: list[str] = []
29     Interpreter(output_callback=outputs.append).execute(tree)
30
31     return PipelineResult(
32         tokens=tokens,
33         ast_text=ast_text,
34         checked_scope=checked_scope,
35         outputs=outputs,
36     )
37
38
39 def format_tokens(tokens: list[Token]) -> str:
40     lines = []
41     for token in tokens:
42         lines.append(
43             f"    {token.token_type.name:<14} {token.lexeme!r:<12} line={
44                 token.line} col={token.column}"
45         )
46     return "\n".join(lines)
47
48
49 def format_stage_output(result: PipelineResult) -> str:
50     sections = [
51         "=" * 60,
52         "  STAGE 1 -- TOKENS",
53         "=" * 60,
54         format_tokens(result.tokens),
55     ]

```

```

54     "",
55     "=" * 60,
56     "    STAGE 2 -- AST (parser output)",
57     "=" * 60,
58     result.ast_text,
59     "",
60     "=" * 60,
61     "    STAGE 3 -- TYPE CHECK",
62     "=" * 60,
63     result.checked_scope.format_table(),
64     "",
65     "=" * 60,
66     "    STAGE 4 -- EXECUTION",
67     "=" * 60,
68     "\n".join(result.outputs) if result.outputs else "(no output)",
69 ]
70 return "\n".join(sections)

```

Listing 8.10: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```

1 from __future__ import annotations
2
3 import unittest
4
5 from components.lexica import Lexer, LexerError
6 from components.parser import Parser, ParseError
7 from components.pipeline import run_pipeline
8 from components.symbol_table import LanguageType, SymbolTable,
9     SymbolTableError
10 from components.tokens import TokenType
11 from components.type_checker import TypeCheckError
12
13 #
14 # -----
15 #
16
17 # Lexer
18 # -----
19
20 class LexerTests(unittest.TestCase):
21     """Tests for components/lexica.py -- lexical analysis and
22     tokenisation."""
23
24     def test_integer_literal_token(self) -> None:
25         tokens = Lexer("42").tokenize()
26         self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
27         self.assertEqual(tokens[0].lexeme, "42")

```

```

25 def test_float_literal_token(self) -> None:
26     tokens = Lexer("3.14").tokenize()
27     self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
28     self.assertEqual(tokens[0].lexeme, "3.14")
29
30 def test_bool_literals_produce_bool_literal_tokens(self) -> None:
31     tokens = Lexer("true false").tokenize()
32     self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
33     self.assertEqual(tokens[0].lexeme, "true")
34     self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
35     self.assertEqual(tokens[1].lexeme, "false")
36
37 def test_keywords_produce_correct_token_types(self) -> None:
38     tokens = Lexer("if else while def return print").tokenize()
39     expected = [
40         TokenType.IF, TokenType.ELSE, TokenType.WHILE,
41         TokenType.DEF, TokenType.RETURN, TokenType.PRINT,
42     ]
43     actual = [t.token_type for t in tokens if t.token_type !=
44               TokenType.EOF]
45     self.assertEqual(actual, expected)
46
47 def test_identifier_not_confused_with_keyword(self) -> None:
48     tokens = Lexer("iffy whileloop").tokenize()
49     self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
50     self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
51
52 def test_two_character_operators(self) -> None:
53     tokens = Lexer("== !=").tokenize()
54     self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
55     self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)
56
57 def test_source_position_tracked_across_lines(self) -> None:
58     tokens = Lexer("x\ny").tokenize()
59     self.assertEqual(tokens[0].line, 1)
60     self.assertEqual(tokens[1].line, 2)
61
62 def test_unexpected_character_raises_lexer_error(self) -> None:
63     with self.assertRaises(LexerError):
64         Lexer("@").tokenize()
65
66 def test_lone_exclamation_raises_lexer_error(self) -> None:
67     with self.assertRaises(LexerError):
68         Lexer("!5").tokenize()
69
70 def test_unterminated_string_raises_lexer_error(self) -> None:
71     with self.assertRaises(LexerError):
72         Lexer('"hello').tokenize()
73
74 #

```

```

75 # Symbol table
76 #
77
78 class SymbolTableTests(unittest.TestCase):
79     """Tests for components/symbol_table.py -- scoped symbol table and
80     semantic types."""
81
82     def test_define_variable_and_lookup(self) -> None:
83         table = SymbolTable()
84         table.define_variable("x", LanguageType.INTEGER, initialized=
85             True)
86         symbol = table.lookup("x")
87         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
88
89     def test_duplicate_variable_definition_raises_error(self) -> None:
90         table = SymbolTable()
91         table.define_variable("x", LanguageType.INTEGER)
92         with self.assertRaises(SymbolTableError):
93             table.define_variable("x", LanguageType.FLOAT)
94
95     def test_duplicate_function_definition_raises_error(self) -> None:
96         table = SymbolTable()
97         table.define_function("f", LanguageType.INTEGER, [])
98         with self.assertRaises(SymbolTableError):
99             table.define_function("f", LanguageType.FLOAT, [])
100
101     def test_lookup_in_parent_scope(self) -> None:
102         parent = SymbolTable()
103         parent.define_variable("x", LanguageType.INTEGER, initialized=
104             True)
105         child = parent.child_scope()
106         symbol = child.lookup("x")
107         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
108
109     def test_undefined_symbol_raises_error(self) -> None:
110         table = SymbolTable()
111         with self.assertRaises(SymbolTableError):
112             table.lookup("z")
113
114     def test_format_table_contains_variable_row(self) -> None:
115         table = SymbolTable()
116         table.define_variable("counter", LanguageType.INTEGER,
117             initialized=True)
118         output = table.format_table()
119         self.assertIn("counter", output)
120         self.assertIn("variable", output)
121         self.assertIn("Integer", output)
122
123     def test_format_table_contains_function_row(self) -> None:
124         table = SymbolTable()

```

```

121         table.define_function("add", LanguageType.INTEGER, [("a",
122             LanguageType.INTEGER)])
123         output = table.format_table()
124         self.assertIn("add", output)
125         self.assertIn("function", output)
126
127     #
128     -----
129
130     # Parser
131     #
132     -----
133
134     class ParserTests(unittest.TestCase):
135         """Tests for components/parser.py -- recursive-descent parsing."""
136
137         def _parse(self, source: str):
138             return Parser(Lexer(source).tokenize()).parse()
139
140         def test_missing_semicolon_raises_parse_error(self) -> None:
141             with self.assertRaises(ParseError):
142                 self._parse("x = 5")
143
144         def test_unclosed_brace_raises_parse_error(self) -> None:
145             with self.assertRaises(ParseError):
146                 self._parse("if (true) { x = 1;}")
147
148         def test_operator_precedence_mul_before_add(self) -> None:
149             # 2 + 3 * 4 must be 14, not 20
150             result = run_pipeline("x = 2 + 3 * 4; print(x);")
151             self.assertEqual(result.outputs, ["14 : Integer"])
152
153         def test_operator_left_associativity(self) -> None:
154             # 10 - 3 - 2 must be 5 (left-associative), not 9
155             result = run_pipeline("x = 10 - 3 - 2; print(x);")
156             self.assertEqual(result.outputs, ["5 : Integer"])
157
158         def test_parentheses_override_precedence(self) -> None:
159             # (2 + 3) * 4 must be 20
160             result = run_pipeline("x = (2 + 3) * 4; print(x);")
161             self.assertEqual(result.outputs, ["20 : Integer"])
162
163     #
164     -----
165
166     # Type checker
167     #
168     -----

```

```

165 class TypeCheckerTests(unittest.TestCase):
166     """Tests for components/type_checker.py -- static type inference and
        checking."""
167
168     def test_integer_arithmetic_stays_integer(self) -> None:
169         result = run_pipeline("x = 3 + 4; print(x);")
170         self.assertEqual(result.outputs, ["7 : Integer"])
171
172     def test_division_always_produces_float(self) -> None:
173         # integer / integer -> Float by design
174         result = run_pipeline("x = 10 / 2; print(x);")
175         self.assertEqual(result.outputs, ["5.0 : Float"])
176
177     def test_float_promotion_in_addition(self) -> None:
178         result = run_pipeline("x = 1 + 2.5; print(x);")
179         self.assertEqual(result.outputs, ["3.5 : Float"])
180
181     def test_string_variable_type_inferred(self) -> None:
182         result = run_pipeline('x = "hello"; print(x);')
183         self.assertEqual(result.outputs, ['"hello" : String'])
184
185     def test_boolean_literal_type_inferred(self) -> None:
186         result = run_pipeline("x = true; print(x);")
187         self.assertEqual(result.outputs, ["true : Boolean"])
188
189     def test_boolean_equality_raises_type_error(self) -> None:
190         # Boolean == Boolean is not allowed; == only accepts arithmetic
191         operands
192         with self.assertRaises(TypeError):
193             run_pipeline('x = true; if (x == false) { print("then"); }'
194                           'else { print("else"); }')
195
196     def test_arithmetic_on_string_raises_type_error(self) -> None:
197         with self.assertRaises(TypeError):
198             run_pipeline('x = "a" + 1;')
199
200     def test_if_non_boolean_condition_raises_type_error(self) -> None:
201         with self.assertRaises(TypeError):
202             run_pipeline("if (1) { x = 2; }")
203
204     def test_while_non_boolean_condition_raises_type_error(self) ->
205         None:
206         with self.assertRaises(TypeError):
207             run_pipeline("x = 0; while (x) { x = x + 1; }")
208
209     def test_assignment_type_mismatch_raises_type_error(self) -> None:
210         with self.assertRaises(TypeError):
211             run_pipeline('x = 5; x = "hello";')
212
213     def test_undefined_variable_raises_type_error(self) -> None:
214         with self.assertRaises(TypeError):
215             run_pipeline("print(z);")

```

```

214 def test_function_wrong_arg_count_raises_type_error(self) -> None:
215     with self.assertRaises(TypeError):
216         run_pipeline("def f(a) { return a; } f(1, 2);")
217
218 def test_function_arg_type_mismatch_raises_type_error(self) -> None:
219     :
220     # first call infers a: Integer; second call with String must
221     # fail
222     with self.assertRaises(TypeError):
223         run_pipeline('def f(a) { return a; } f(1); f("hello");')
224
225 def test_uncalled_function_body_type_error_is_caught(self) -> None:
226     # type error inside body must be detected even if function is
227     # never called
228     with self.assertRaises(TypeError):
229         run_pipeline('def bad() { y = "hello" + 1; }')
230
231 def test_valid_uncalled_function_does_not_raise(self) -> None:
232     # a well-typed but uncalled function should not raise
233     result = run_pipeline("def add(a, b) { return a + b; }")
234     self.assertEqual(result.outputs, [])
235
236 #
237
238 # Integration (full pipeline)
239 #
240
241 -----
242
243 class IntegrationTests(unittest.TestCase):
244     """End-to-end tests through all four pipeline stages."""
245
246     # --- Unary minus ---
247
248     def test_unary_minus_integer(self) -> None:
249         result = run_pipeline("x = -5; print(x);")
250         self.assertEqual(result.outputs, ["-5 : Integer"])
251
252     def test_unary_minus_float(self) -> None:
253         result = run_pipeline("y = -3.14; print(y);")
254         self.assertEqual(result.outputs, ["-3.14 : Float"])
255
256     def test_unary_minus_variable(self) -> None:
257         result = run_pipeline("x = 10; y = -x; print(y);")
258         self.assertEqual(result.outputs, ["-10 : Integer"])
259
260     def test_unary_minus_inside_expression(self) -> None:
261         result = run_pipeline("x = 3 + -2; print(x);")
262         self.assertEqual(result.outputs, ["1 : Integer"])
263
264     # --- Control flow ---

```

```

260
261 def test_if_then_branch_executes(self) -> None:
262     result = run_pipeline(
263         'x = 5; if (x == 5) { print("yes"); } else { print("no"); }
264         ,
265     )
266     self.assertEqual(result.outputs, ['"yes" : String'])
267
268 def test_if_else_branch_executes_when_condition_false(self) -> None:
269     result = run_pipeline(
270         'x = 0; if (x == 5) { print("yes"); } else { print("no"); }
271         ,
272     )
273     self.assertEqual(result.outputs, ['"no" : String'])
274
275 def test_while_loop_executes_repeatedly(self) -> None:
276     result = run_pipeline("x = 3; while (x != 0) { print(x); x = x
277         - 1; }")
278     self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer",
279         "1 : Integer"])
280
281 # --- Functions ---
282
283 def test_function_integer_return(self) -> None:
284     result = run_pipeline("def square(n) { return n * n; } print(
285         square(7));")
286     self.assertEqual(result.outputs, ["49 : Integer"])
287
288 def test_function_type_inference(self) -> None:
289     result = run_pipeline(
290         "def add(a, b) { return a + b; } result = add(2, 3.5);
291         print(result);"
292     )
293     self.assertEqual(result.outputs, ["5.5 : Float"])
294     self.assertIn("result          variable   Float", result.
295         checked_scope.format_table())
296
297 def test_function_value_parameter_passing(self) -> None:
298     # Modifying a parameter inside a function must not affect the
299     # caller's variable
300     result = run_pipeline(
301         "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x);
302         print(x); print(y);"
303     )
304     self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer
305         "])
306
307 def test_nested_function_calls(self) -> None:
308     result = run_pipeline(
309         "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
310     )
311     self.assertEqual(result.outputs, ["6 : Integer"])

```

```

302
303     # --- print() with all four types ---
304
305     def test_print_integer(self) -> None:
306         result = run_pipeline("print(42);")
307         self.assertEqual(result.outputs, ["42 : Integer"])
308
309     def test_print_float(self) -> None:
310         result = run_pipeline("print(3.14);")
311         self.assertEqual(result.outputs, ["3.14 : Float"])
312
313     def test_print_boolean(self) -> None:
314         result = run_pipeline("print(true);")
315         self.assertEqual(result.outputs, ["true : Boolean"])
316
317     def test_print_string(self) -> None:
318         result = run_pipeline('print("plc");')
319         self.assertEqual(result.outputs, ['"plc" : String'])
320
321     # --- Reassignment ---
322
323     def test_variable_can_be_reassigned_same_type(self) -> None:
324         result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
325         self.assertEqual(result.outputs, ["3 : Integer"])
326
327     def test_scope_isolation_if_block(self) -> None:
328         # Variable defined inside an if-block must not be visible
329         outside it
330         with self.assertRaises(TypeError):
331             run_pipeline(
332                 "x = 1; if (x == 1) { inner = 99; } print(inner);"
333             )
334
335 if __name__ == "__main__":
336     unittest.main()

```

Listing 8.11: tests/test_pipeline.py — automated regression suite (54 tests)

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Assignment Scope
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment execution; if execution; while execution; function execution; <code>print()</code> execution; unary minus type inference and execution; pipeline integration (<code>pipeline.py</code>); automated test suite (<code>test_pipeline.py</code> , 54 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (character scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords and operators; identifiers and literals; variable/function storage; type storage
Win Htut Naing	st126687	Arithmetic expressions; boolean expressions; assignment statements; if-then-else; while-loop; function definitions; function calls; <code>print()</code> syntax; unary minus parsing